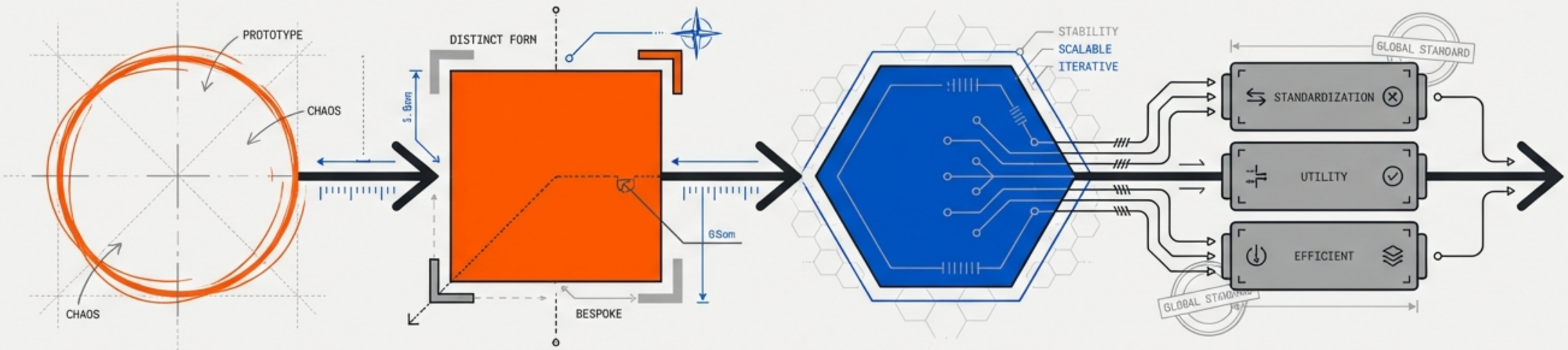
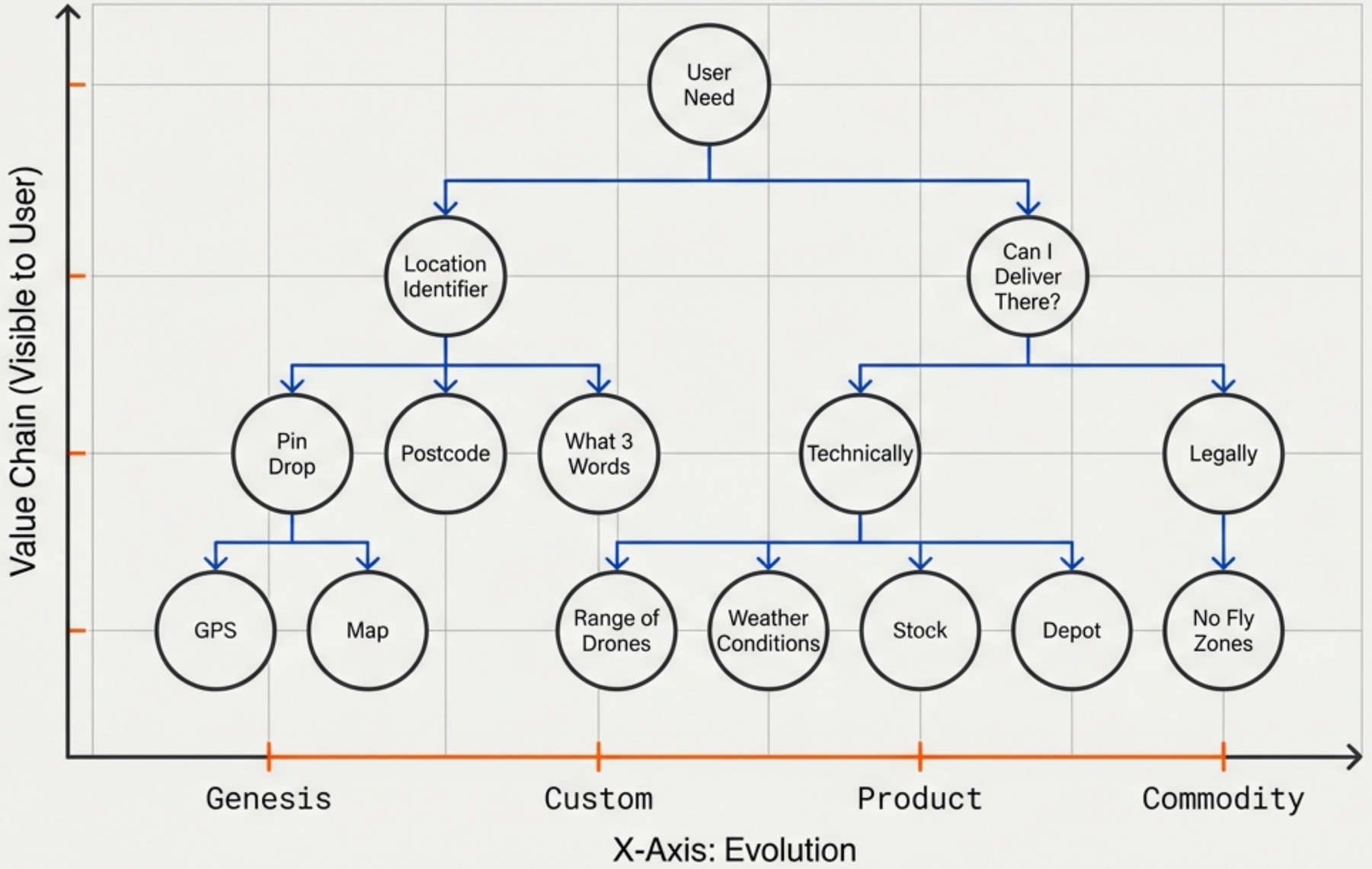
STRATEGIC SOFTWARE EVOLUTION: FROM GENESIS TO COMMODITY

Adapting Wardley Maps to engineer better systems, manage technical debt, and accelerate development with LLMs.



Producer's Note
Strategy / Engineering / AI

Systems are not static; they drift from novel ideas to invisible utilities



CORE CONCEPTS

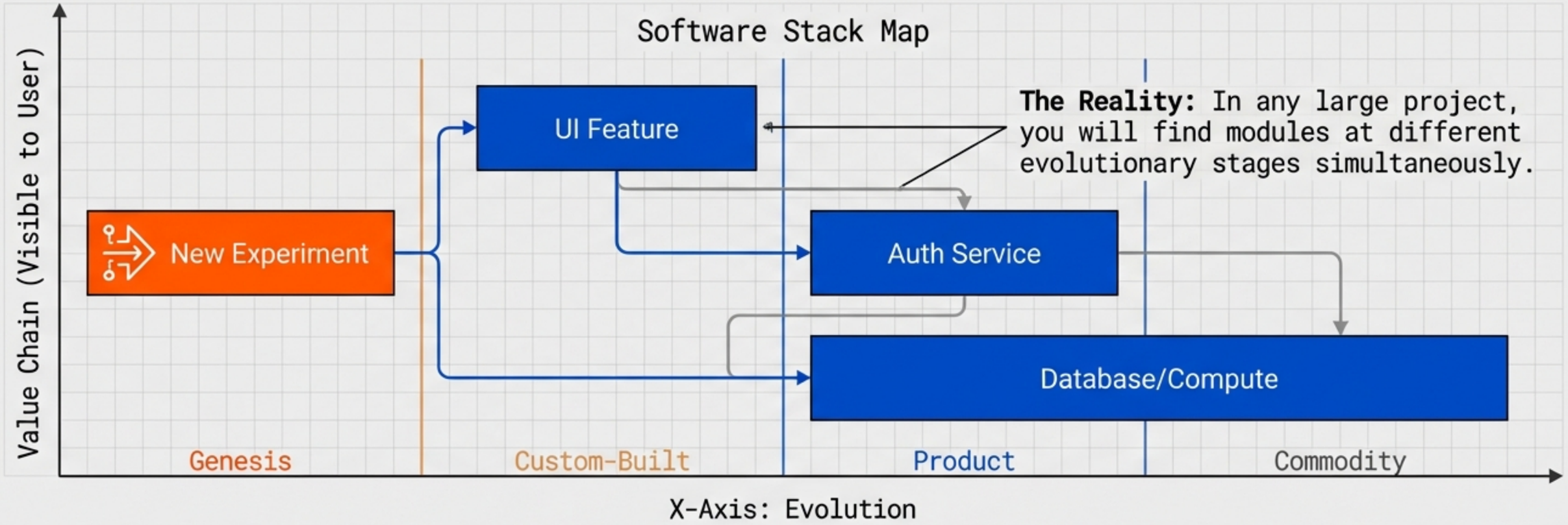
- **The Y-Axis:** Components are anchored by User Needs. Every component exists to serve a need above it.
- **The X-Axis:** Components move from left to right as they mature.
- **The Premise:** A system is a value chain of components, ranging from 'Genesis' (uncertain/new) to 'Commodity' (standard/ubiquitous).

Translating the four stages of evolution into code

GENESIS	CUSTOM-BUILT	PRODUCT	COMMODITY
			
Definition: Novel, experimental, high risk.	**Definition:** Bespoke, evolving, scarce.	**Definition:** Stable, well-defined, competitive.	**Definition:** Standardised, ubiquitous, low-cost.
Code Equivalent: Prototypes, hacky scripts, 'frontier' code.	**Code Equivalent:** Unique internal features, solutions crafted specifically for the product.	**Code Equivalent:** Mature internal libraries, polished services with stable APIs.	**Code Equivalent:** Open-source frameworks, cloud services (AWS/Azure), utilities.

Maturation over time

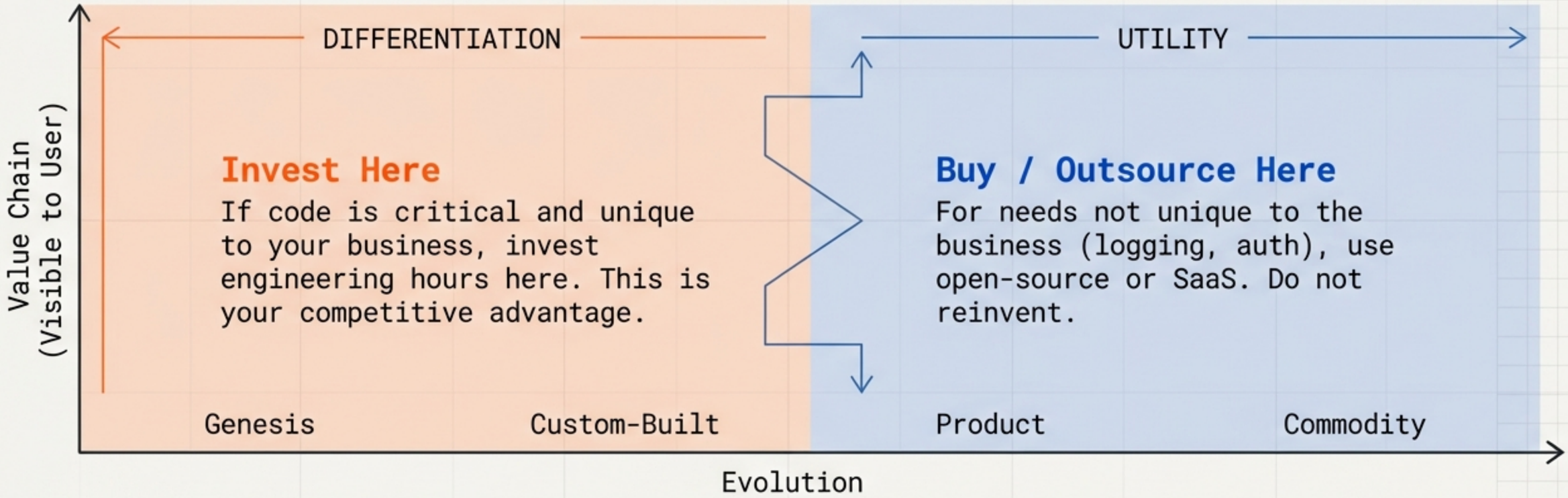
Your codebase is a map of mixed maturities



The Trap: Treating a 'Genesis' prototype as if it were a stable 'Product' leads to premature optimisation. Treating a 'Commodity' as 'Custom' leads to reinventing the wheel.

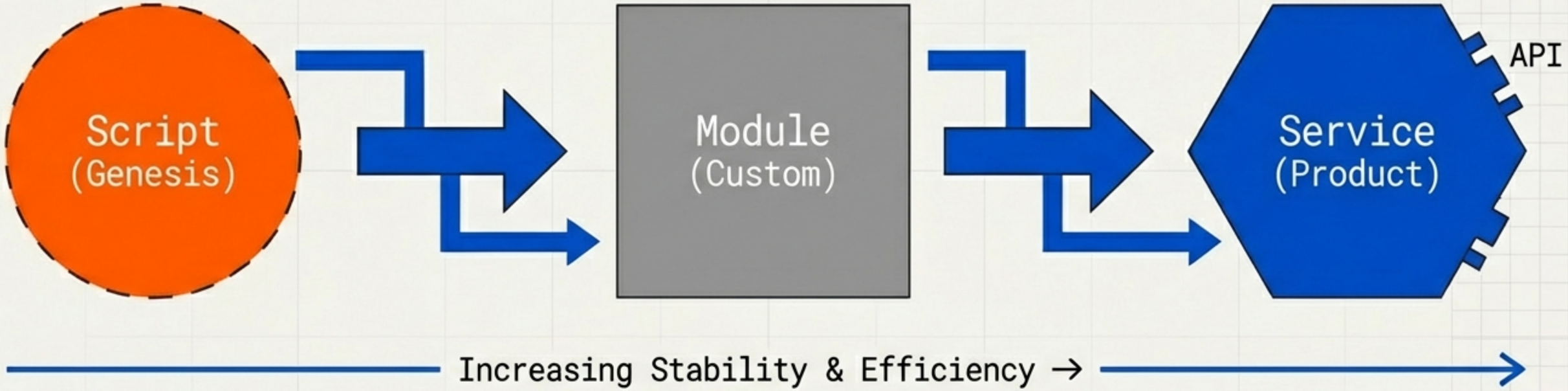
By mapping your software components, you gain insight into where you are doing pioneering work versus using well-established solutions.

Focus engineering effort on the places of maximum yield



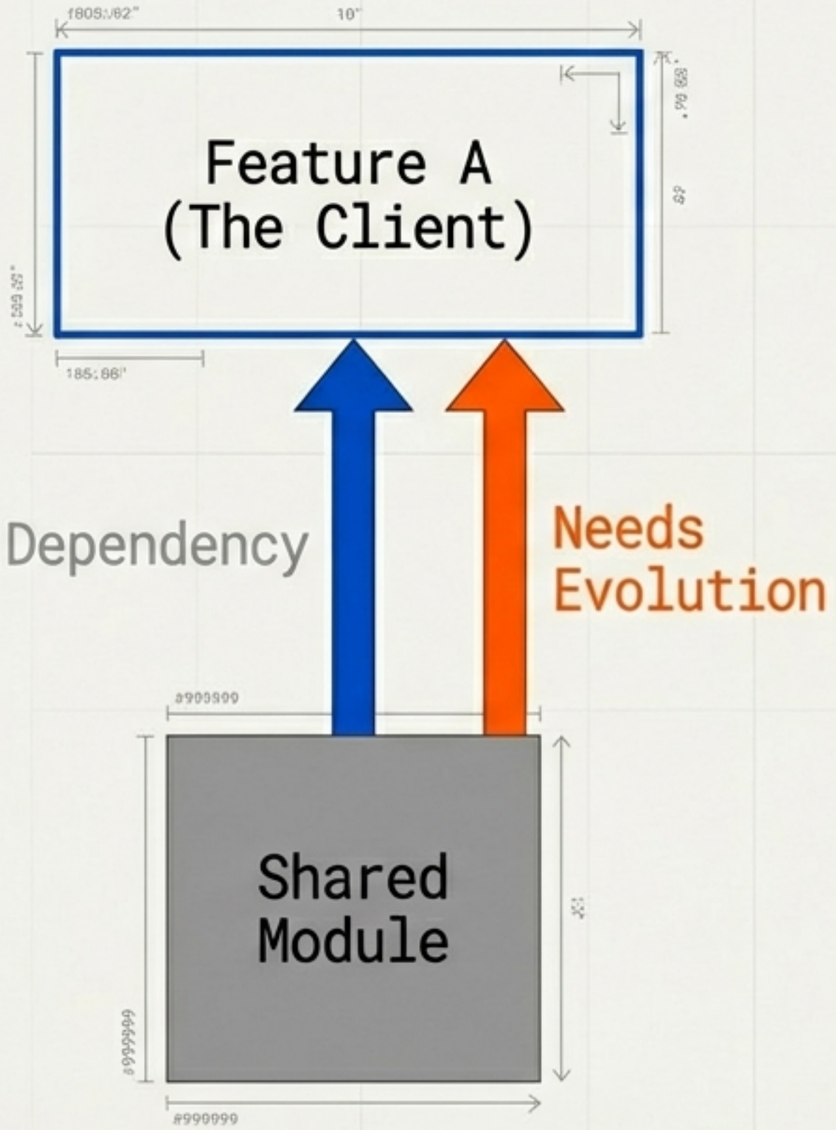
“Concentrate your limited development time on what truly differentiates your software.”

Refactoring is the act of moving components to the right



Each refactor is an investment to move a component to a higher stage of stability. Maturing ‘custom-built’ code into ‘product grade’ creates reusable assets, turning a one-off solution into a toolkit of proven components.

Never evolve a component without a client



The Principle:
Make sure there is a user need (or a higher-level component) pulling for that evolution.

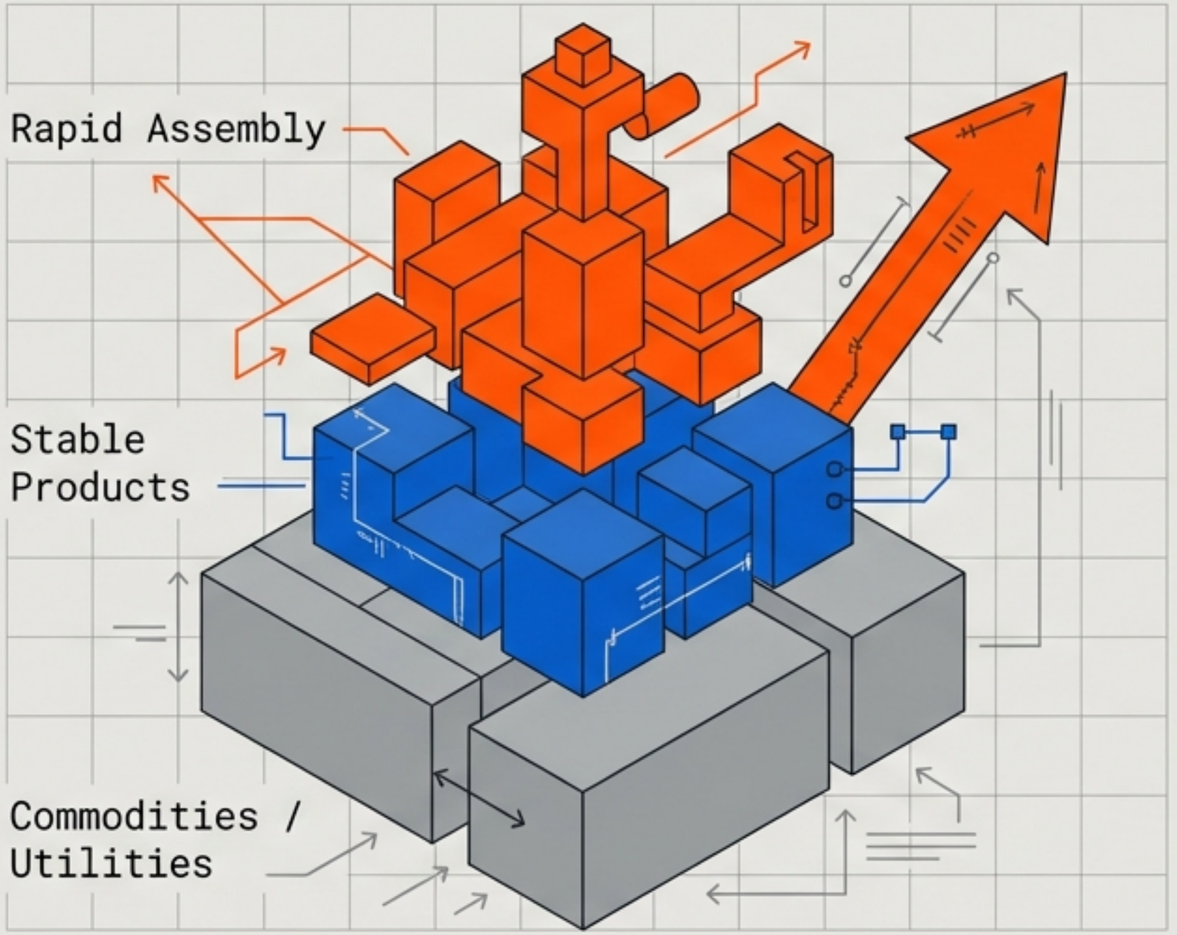
Connection to YAGNI:
“You Aren’t Gonna Need It.” Refactoring for theoretical purity is waste.

Actionable Advice:
Treat every refactor as a mini-product development.

Bad: Rewriting an internal function that works fine and isn’t reused.

Good: Extracting a common module because three different features need to parse data the same way.

The counter-intuitive link between code volume and velocity



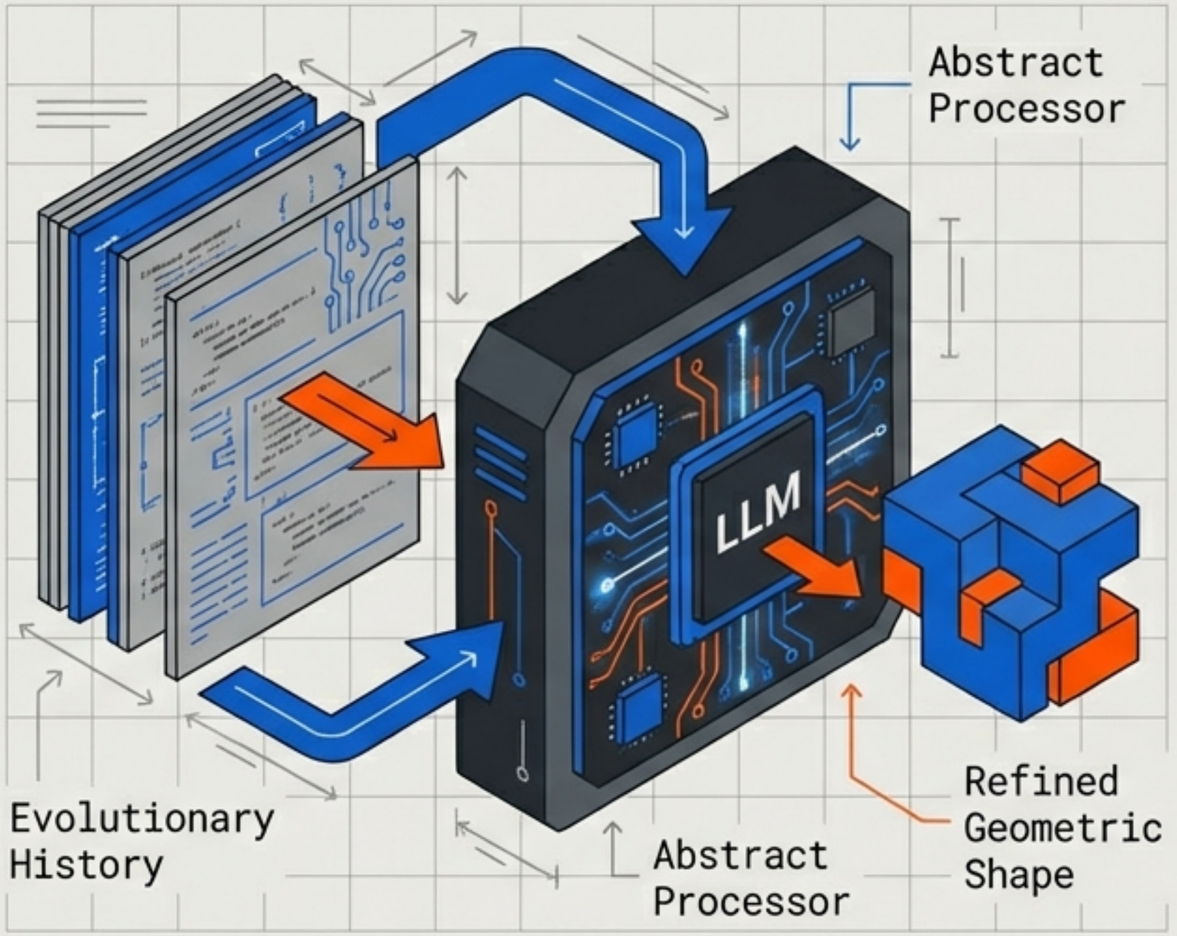
The Paradox: Having MORE code (in the form of well-factored, stable modules) makes you faster, not slower.

Industrialisation = Acceleration

As components become commodities, they enable higher-level innovation to happen faster.

You don't have to reinvent the wheel—you assemble new features rapidly on top of certainty.

Context Engineering: The new dimension of evolutionary design



Feeding Large Language Models (LLMs) not just a static spec, but the **history** of a module's evolution.

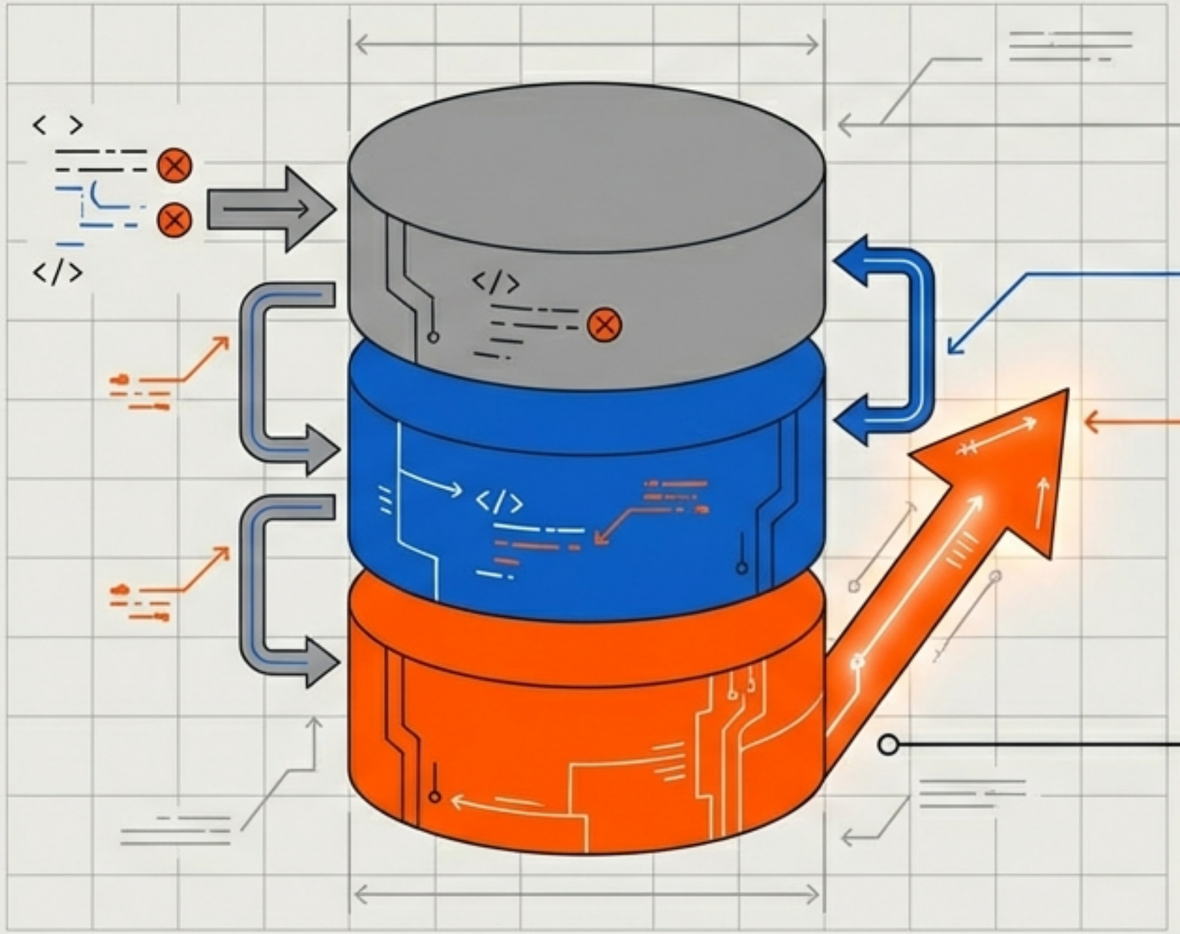
The Shift:

- From: "Write a function that does X"
- To: "Here is the evolution of this module, help us take the next step."

Context Engineering:

Carefully curating the history provided to the AI to produce relevant output.

The most effective AI brief is a history of failure



Prompt Structure

V1 Context: The original code and its specific failures.

V2 Context: The attempt to fix it, and why it was suboptimal.

The Goal: Clean-slate V3 based on lessons learned.

Why It Works

1. Explains the 'Why' behind design decisions.
2. Focuses the AI's finite context window on relevant changes.
3. Prevents the AI from repeating past mistakes by showing them explicitly.

Mitigating the catastrophic risk of the rewrite

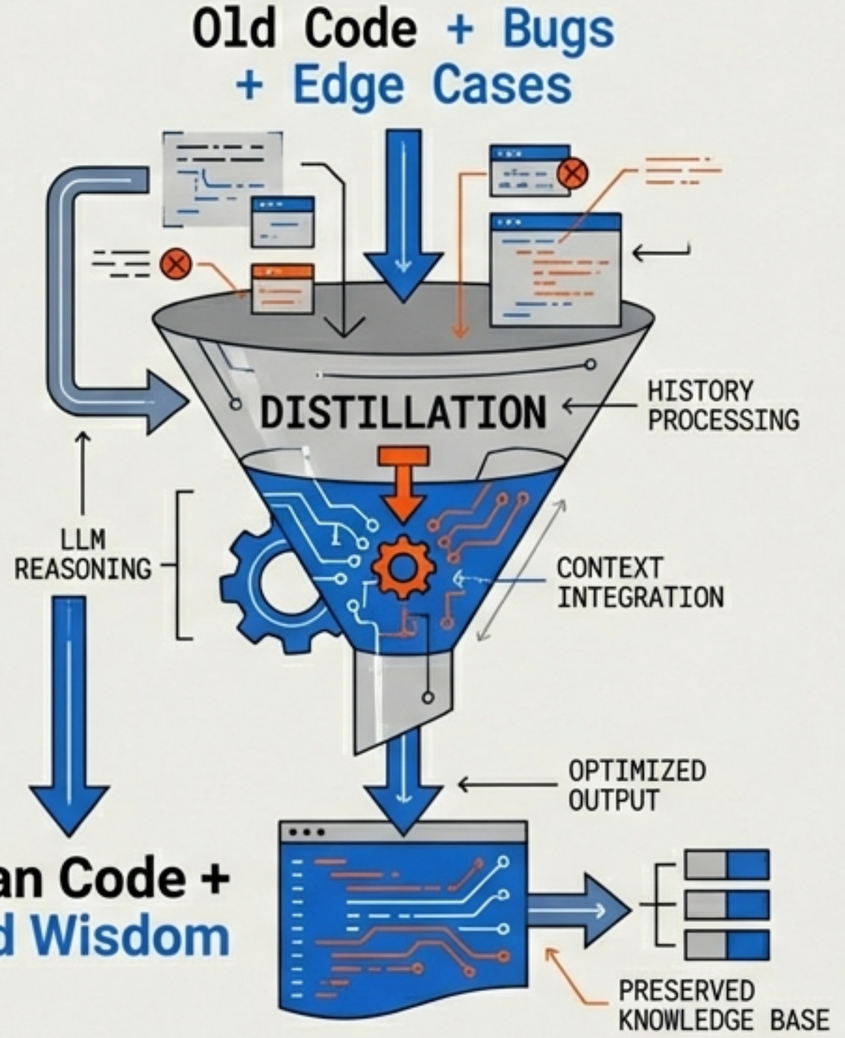
The Old Wisdom

> "When you throw away code and start from scratch, you are throwing away all that knowledge... Years of programming work."
>
> — Joel Spolsky

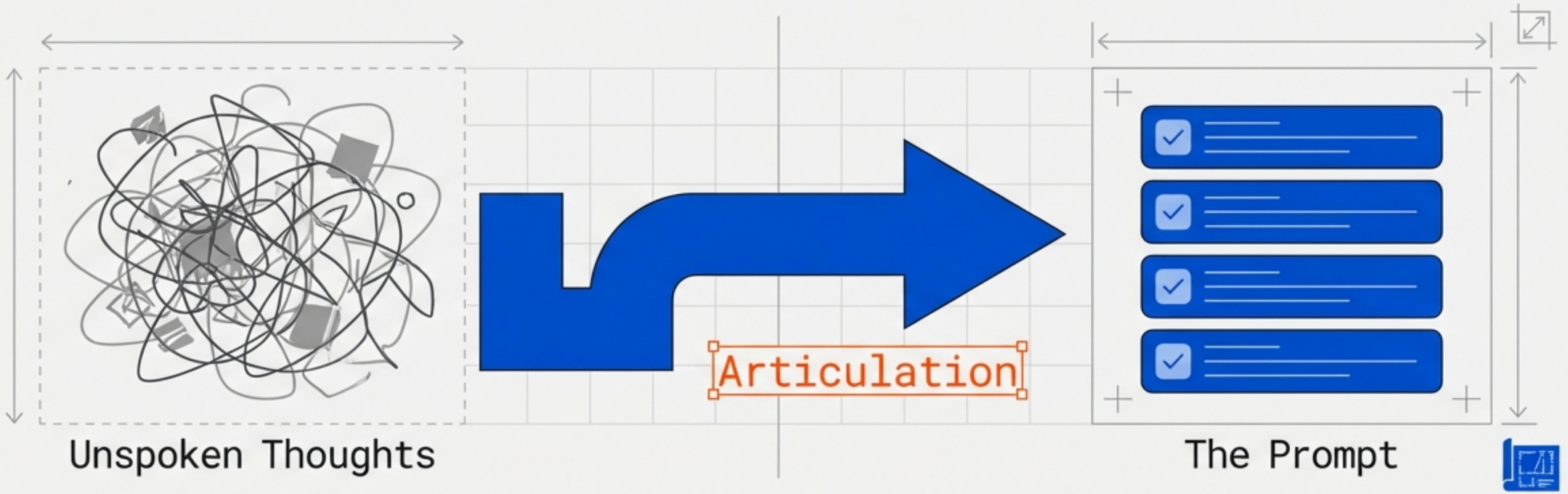
The New Reality

Guided Regeneration

By including the old code and its history in the LLM prompt, you **distill** that knowledge rather than discarding it.



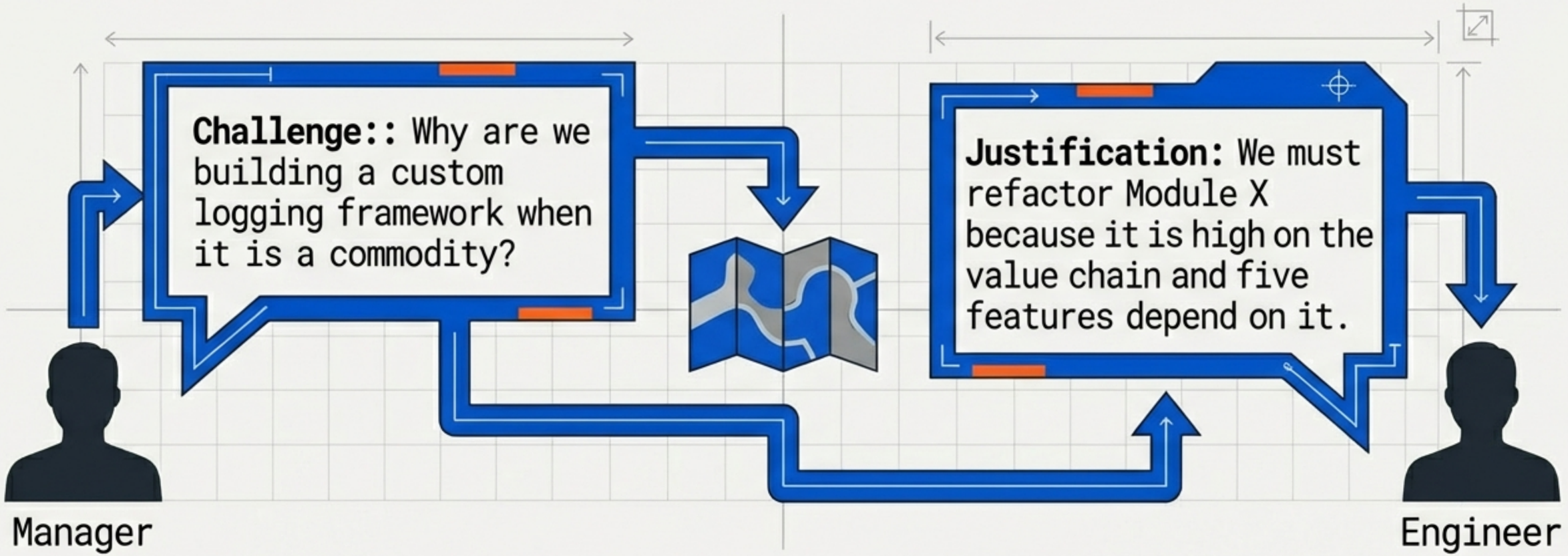
The hidden value of the prompt as a design document



The Rubber Duck Effect
To write a good evolutionary prompt, you must clearly explain what the component does. This forces a moment of reflection.

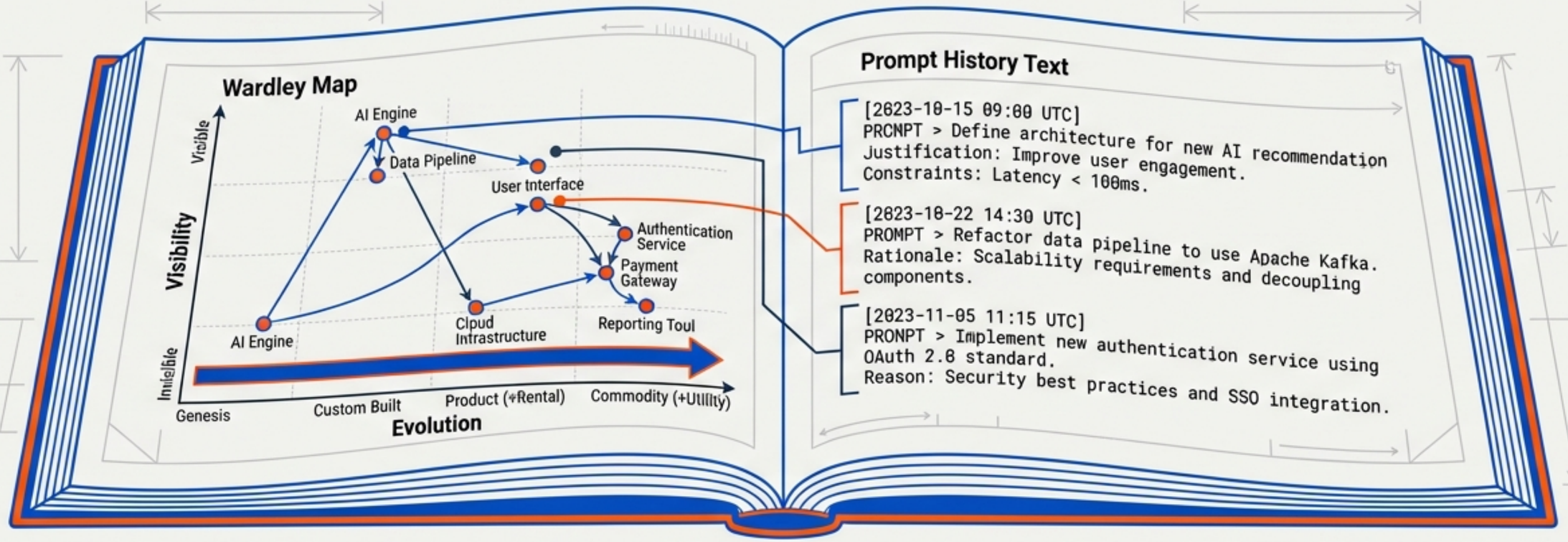
Outcome:
The prompt serves as a thorough design document and rationale, clarifying thinking before a single line of new code is generated.

Strategic clarity drives technical decision making



Benefit: The map provides an objective rationale for architectural decisions, moving debates away from opinion and toward strategy.

Living documentation facilitates faster onboarding



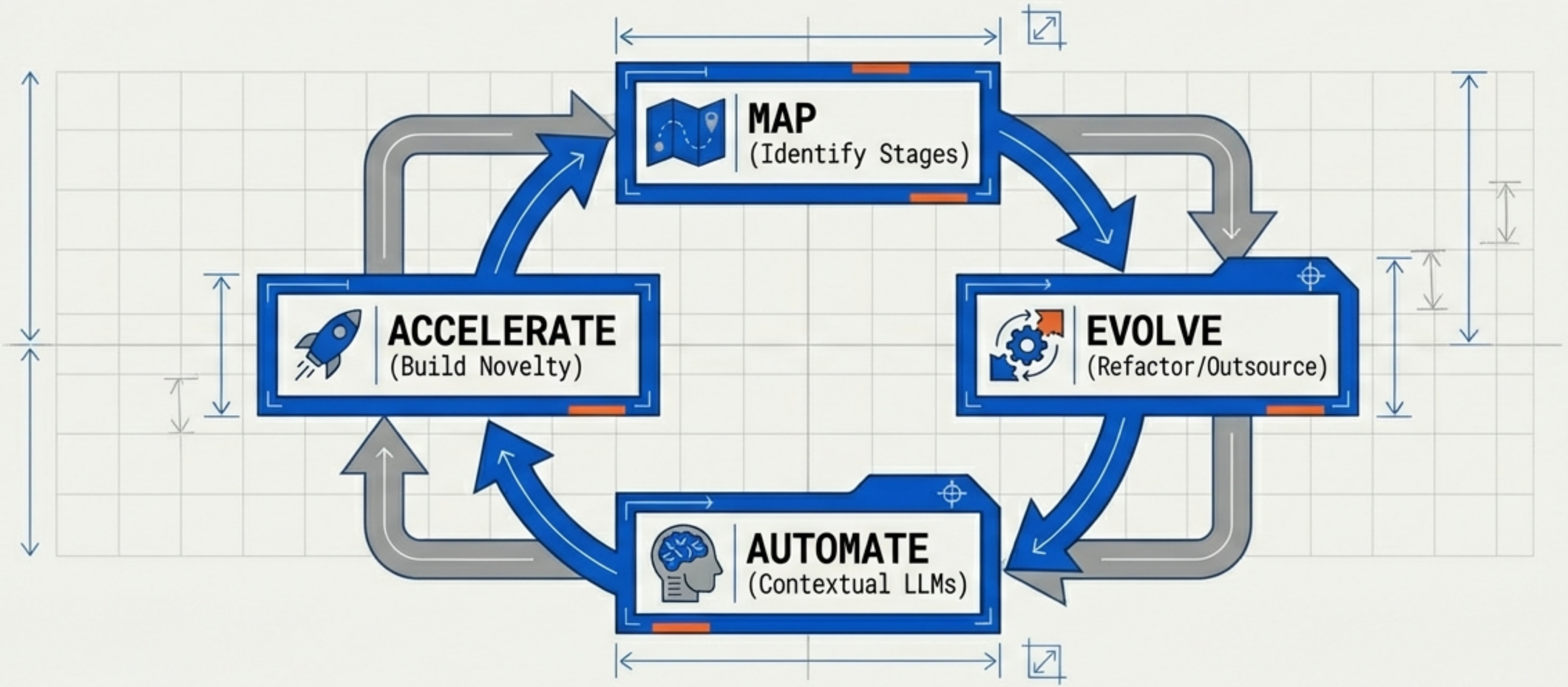
The Problem: New engineers often struggle to understand *why* the system is built the way it is.

The Solution:

1. **The Map:** Gives a mental model of the landscape (what is core vs. standard).
2. **The History Prompts:** Serve as narratives explaining the journey of specific components.

Result: Seamless knowledge transfer.

The feedback loop of high-velocity engineering



The more code and experience you have, the faster you should go.
Each evolutionary step builds a stronger foundation for the next.